



# 南京大學

## 本科畢業論文

|      |                    |
|------|--------------------|
| 学 院  | 智能软件与工程学院          |
| 专 业  | 软件工程（智能化软件）        |
| 题 目  | 实践中的理论最优哈希表        |
| 年 级  | 2022 学 号 221900323 |
| 学生姓名 | 纪泽操                |
| 指导教师 | 刘明谋 职 称 副教授        |
| 提交日期 | 2026 年 5 月 20 日    |

# 南京大学本科生毕业论文（设计、作品）中文摘要

**题目：**实践中的理论最优哈希表

**学院：**智能软件与工程学院

**专业：**软件工程（智能化软件）

**本科生姓名：**纪泽操

**指导教师（姓名、职称）：**刘明谋 副教授

**摘要：**

开放寻址哈希表在工程里很常见，但在高负载下既要省内存、又要控制插入删除的扰动，并不容易。Bender 等人的论文《On the Optimal Time/Space Tradeoff for Hash Tables》给出了一条理论路线：用有限次“踢键”换更紧凑的表示。本毕业设计以该论文为主要参照，用 C++ 写了一个可运行的原型：键先经可逆置换再分区；每个 Facility 内用 Router 做局部定位；槽里做商压缩，变长恢复信息放在 MiniArray；扩缩容采用新旧两张表逐步搬迁。

实现上对原文有多处简化：k-kick 只在单个 tail cubby 里用有界随机探针和踢出链近似；MiniArray 与 Router 在更新时允许整段重打包或重建 trie，以保证代码清晰和指标好记。功能测试覆盖插入、查询、删除和重复插入；另有独立微基准查看热路径延迟。实验表明，在默认参数下查询延迟稳定在纳秒量级，插入阶段因重打包和扩容会慢一些。本工作重在把论文里的主要模块串起来并说明与理论的差距，而不是证明渐近最优性。

**关键词：**哈希表；时间空间权衡；探针复杂度；增强开放寻址；k-kick tree；元数据编码；动态可扩容；succinct 数据结构

# 南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: On the Optimal Time/Space Tradeoff for Hash Tables

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software Engineering (Intelligent Software)

UNDERGRADUATE: Ji Zecao

MENTOR: Liu Mingmou, Associate Professor

ABSTRACT:

This undergraduate thesis implements a simplified prototype of the hash table construction in “On the Optimal Time/Space Tradeoff for Hash Tables.” The code uses a reversible permutation for routing, per-facility routers to locate cubbies and slots, quotient-compressed payloads with variable-length metadata, and two-table incremental resizing.

Several parts of the theory are only approximated: k-kick style behavior is reduced to a bounded kick chain inside the tail cubby, and metadata structures favor straightforward updates over the tightest asymptotics. Functional tests check correctness; a small micro-benchmark reports latency and counters. The goal is a working, observable system and a clear statement of engineering tradeoffs, not a full reproduction of every formal guarantee in the paper.

KEYWORDS: hash tables; time–space tradeoff; probe complexity; enhanced open addressing; k-kick tree; metadata encoding; dynamic expansion; succinct data structures

# 目 录

|                               |     |
|-------------------------------|-----|
| 目 录                           | III |
| 第一章 引言                        | 1   |
| 1.1 研究背景及意义                   | 1   |
| 1.2 国内外研究现状                   | 1   |
| 1.3 研究路线                      | 2   |
| 第二章 问题模型与理论基础                 | 3   |
| 2.1 探针复杂度与球槽分配模型              | 3   |
| 2.2 k-kick 树分层驱逐思想            | 3   |
| 2.3 紧凑元数据存储核心思想               | 4   |
| 2.4 分层架构整体设计思想                | 4   |
| 2.5 商压缩的信息论最优思想               | 5   |
| 2.6 算法层次全景总结                  | 5   |
| 第三章 系统设计与实现                   | 6   |
| 3.1 总体架构与数据路径                 | 6   |
| 3.2 参数派生与地址拆分                 | 7   |
| 3.3 Facility 组件               | 8   |
| 3.4 Cubby、Slot 与 FreeSlotTree | 8   |
| 3.5 MiniArray 组件              | 9   |
| 3.6 Router 组件                 | 10  |
| 3.7 MetaEntry 与商压缩            | 11  |
| 3.8 插入路径与简化 k-kick            | 12  |
| 3.9 查询与删除路径                   | 13  |

|                                |           |
|--------------------------------|-----------|
| 3.10 扩缩容与双表渐进迁移 . . . . .      | 14        |
| <b>第四章 系统验证与实验设计 . . . . .</b> | <b>16</b> |
| 4.1 实验目标与评价口径 . . . . .        | 16        |
| 4.2 功能正确性测试 . . . . .          | 16        |
| 4.3 性能基准 . . . . .             | 17        |
| 4.4 指标与数据库观测 . . . . .         | 18        |
| <b>第五章 总结与展望 . . . . .</b>     | <b>19</b> |
| 5.1 工作总结 . . . . .             | 19        |
| 5.2 不足与局限 . . . . .            | 19        |
| 5.3 可改进方向 . . . . .            | 20        |
| <b>参考文献 . . . . .</b>          | <b>21</b> |
| <b>致 谢 . . . . .</b>           | <b>22</b> |

# 第一章 引言

## 1.1 研究背景及意义

哈希表通过哈希函数将关键字映射到数组位置，是数据库索引、键值存储、编译器符号表和运行时系统中常用的基础结构。开放寻址哈希表具有连续存储和查询路径短等优点，但在高负载下容易受到冲突影响。工程实现通常通过预留空槽降低冲突概率，这会带来额外空间消耗。因此，哈希表设计中的一个核心问题是：在保持操作时间可控的前提下，如何减少单个元素对应的附加空间。

论文 *On the Optimal Time/Space Tradeoff for Hash Tables* 对这一问题给出了理论化刻画<sup>[1]</sup>。该文指出，在常数时间查询的约束下，若允许插入和删除操作承担  $O(k)$  的元素交换成本，则每个元素的额外空间可由  $O(\log \log n)$  比特降低到  $O(\log^{(k)} n)$  比特<sup>12</sup>。该结论说明，哈希表并非只能在“低负载高空间”和“高负载高冲突”之间取舍，还可以通过受控迁移、紧凑路由和商压缩等机制重新分配时间与空间成本。本文在该理论工作的基础上实现一个 C++ 原型，目的在于把论文中的主要结构落到可运行系统中，并通过测试和微基准观察其核心路径的行为。

## 1.2 国内外研究现状

哈希表的经典实现主要包括链地址法和开放寻址法。链地址法通过链表或桶结构处理冲突，实现简单但指针和分配开销较明显；开放寻址法将元素直接放入数组槽位，缓存局部性较好，但性能对负载因子和探针序列较敏感<sup>[2]</sup>。在此基础上，布谷鸟哈希通过多个候选位置和踢出过程提高查询确定性<sup>[3]</sup>，完美哈希则面向静态集合给出更强的查找保证<sup>[4]</sup>。这些方法各自侧重查询效率、构造代价或空间利用率，但动态场景下的空间下界仍然是较难同时兼顾的问题。

---

1 本文中讨论的空间开销均以比特为单位。

2 记号  $\log^{(k)} n$  表示对  $n$  连续取  $k$  次对数： $\log \log \cdots \log n$ （共  $k$  个  $\log$ ）。

近年来，相关研究更加关注哈希表的位级空间开销。与只比较装载率不同，位级分析要求明确记录每个元素为支持查询、更新和恢复所额外保存的信息量。本文参考的最优哈希表理论正是在这一背景下提出：它将元素放置抽象为球槽分配问题，并用 k-kick 树、Local Query Router、MiniArray 和商压缩组合出新的时间—空间权衡。本文不试图重新证明该理论结果，而是围绕其中可工程化的结构进行实现与验证。

### 1.3 研究路线

本文尝试落地论文 *On the Optimal Time/Space Tradeoff for Hash Tables* 中的主要结构。系统使用 Facility--Cubby--Slot 组织局部存储；每个 Facility 维护一组 Router，用于把键定位到具体 Cubby 和槽位；槽位中保存商压缩后的 payload，MiniArray 保存恢复键所需的变长元数据；扩缩容采用 active/old 双表渐进迁移。实现侧重点是打通插入、查询、删除、键恢复和迁移路径，并记录踢出次数、路由步数、元数据位数等指标。

本文围绕三个问题展开：其一，原型系统能否在批量插入、命中查询、删除、重复插入和迁移窗口中保持一致语义；其二，实际运行中插入移动次数、Router 访问步数和逻辑元数据位数呈现何种特征；其三，当前 C++ 实现相对于论文结构做了哪些简化，这些简化对实验结论的解释有何限制。第二章说明相关理论基础，第三章给出系统实现，第四章报告测试与微基准结果，第五章总结本文完成的工作和后续可改进方向。

## 第二章 问题模型与理论基础

### 2.1 探针复杂度与球槽分配模型

开放寻址哈希表可以抽象为在线球槽分配问题。设表中有固定数量的槽位，每个槽至多容纳一个元素；元素支持动态插入和删除，任意时刻的元素数不超过槽位数。对每个键  $x$ ，哈希过程为其生成一条固定随机探针序列  $h_1(x), h_2(x), \dots$ ，元素只能被放置在该序列给出的候选槽位中。

若元素最终存放在其探针序列的第  $i$  个候选位置，则探针复杂度定义为  $\Theta(1 + \log i)$ 。该量可理解为查询时记录“使用第几个候选位置”所需的比特数，因此直接对应元数据的空间开销。该模型下有两个需要同时控制的量：平均探针复杂度决定整体附加空间，交换成本表示一次更新中被搬移的元素数量。最优哈希表理论的关键问题，就是在允许有限交换的条件下降低平均探针复杂度。

### 2.2 k-kick 树分层驱逐思想

k-kick 树不是必须以指针树形式存在的数据结构，而是一种分层放置与受控驱逐规则。它将槽位划分为  $k + 1$  个逻辑层级，不同层级的 Cubby 尺寸随深度递减。深层 Cubby 更靠近元素的随机偏好位置，因此具有更高放置优先级；浅层 Cubby 范围更大，用于接收在深层无法放置的元素。

插入前，哈希过程为元素确定一条从深层到浅层的 Cubby 谱系，并确定其允许使用的最大深度。插入时优先尝试深层 Cubby；若目标 Cubby 已满但尚未饱和，则可以驱逐其中优先级较低的元素，将当前元素放入该位置，被驱逐元素再向更浅层寻找合法位置。由于每次驱逐都沿层级向上推进，级联过程受到层数限制。

层数由参数  $k$  控制，因此单次更新中的迁移次数可限制在  $O(k)$  量级。理论结果表明，在这一交换成本下，平均探针复杂度可降至  $O(\log^{(k)} n)$ 。这一结论是后续系统实现中记录 `kick_count`、区分理论 k-kick 与工程版踢出链的依据。

## 2.3 紧凑元数据存储核心思想

分层放置降低了探针复杂度，但查询仍需要知道元素最终所在的 Cubby 和槽位。理论结构为此引入 MiniArray 与 Local Query Router 两类组件，分别处理变长元数据存储和局部键到槽位的映射。

MiniArray 用于保存 Cubby 内不同长度的元数据条目。理论方案利用内存 B 树和查表方法，将变长比特串打包存储，并支持均摊常数时间读写。其空间开销与条目总比特数线性相关，避免为所有槽位按最大长度预留空间。

Local Query Router 负责小规模键集合的本地定位。它将键映射为随机比特串，用二进制前缀树区分不同键，再把树结构和叶子上的槽位信息编码为连续比特流。由于每个 Router 只管理局部少量元素，前缀树的期望规模较小，可在低空间开销下完成键到 Cubby、Slot 的映射。本文第三章中的 Router 和 MiniArray 均参照这一接口语义实现，但更新策略采用了工程化简化。

## 2.4 分层架构整体设计思想

本文实现参照 Facility-Cubby-Slot 三级架构组织数据。Facility 负责表的局部分区与路由，Cubby 负责局部存储和元数据维护，Slot 对应实际数组位置。

Cubby 是局部存储单元，内部包含存储槽、变长元数据和空闲槽检索结构，承担元素放置、局部驱逐和键恢复所需信息维护。Facility 聚合多个 Cubby，并通过局部 Router 将键定位到目标 Cubby 与槽位。理论结构还引入多 tier Cubby 和 tail 机制：大部分非 tail Cubby 尽量保持满载，插入和删除造成的波动集中由 tail 承担。

为适配元素数量变化，理论结构用分布不变量约束各 tier Cubby 的数量，并通过后台拆分、合并平摊重建代价。本文实现保留 Facility、tier、tail、Router 和双表迁移等主要接口，但未完整实现论文中的全部后台重建规则。因此，第二章描述的是理论依据，第三章将进一步说明当前代码中的实现边界。

## 2.5 商压缩的信息论最优思想

若哈希表在槽位中完整保存原始键，则其中一部分信息与寻址过程重复。商压缩利用随机置换后的低位参与路由这一事实，只在槽位中保存无法由位置和元数据恢复的剩余位。

具体而言， $\pi(x)$  的低  $\log N$  位用于确定 Facility、Router 桶和局部偏好位置；槽位只保存去掉这部分后的 payload。为了恢复完整  $\pi(x)$ ，元数据记录偏好槽位与实际槽位之间的差值，以及局部桶内的中间位。结合 Facility 编号、槽位下标和 payload，即可重建置换值，再通过  $\pi^{-1}$  得到原键。第三章将按照当前代码中的 payload、mid\_bits 和 delta 说明该过程。

## 2.6 算法层次全景总结

上述理论结构可归纳为三层：

- Cubby 层负责元素放置、空闲槽检索、局部驱逐和变长元数据编码；
- Router 层负责把键映射到目标 Cubby 与 Slot，并为查询提供局部定位信息；
- Facility 层负责组织 Cubby、维护 tail、承载扩缩容和重建过程中的局部管理。

本文后续实现以这三层为结构参照，同时明确区分理论结构和工程原型之间的差异。

## 第三章 系统设计与实现

系统以 HashTable 为统一入口，内部围绕 Facility--Cubby--Slot 三层结构组织数据，并通过可逆置换、局部 Router、MiniArray 元数据、商压缩和双表渐进迁移完成插入、查询、删除与扩缩容。本章不重复第二章的理论证明，而是对应代码说明各组件包含的状态、实际采用的实现策略，以及这些策略与理论模型之间的差异。

### 3.1 总体架构与数据路径

系统在逻辑上可以看作一个长度为  $N$  的开放寻址表，其中  $N$  始终取为 2 的幂。初始化时，系统根据外部规模提示  $n$  推导表规模  $N$ ，再由 `choose_K` 得到每个 Facility 内的局部桶数  $K$ 。整个表被划分为  $\frac{N}{K}$  个 Facility，每个 Facility 管理约  $K$  个全局地址位置，并在内部维护一组 Router、若干分层 Cubby 以及一个 tail Cubby。Cubby 是实际存储单元，Slot 则是 Cubby 内的单个数组位置。

从操作路径看，系统首先对键  $k$  计算可逆置换值  $g_x = \pi(k)$ ，再用  $g_x$  的低  $\log N$  位确定 Facility 编号与 Router 桶号。Router 负责把键映射到具体的  $(\text{Cubby}^*, \text{slot})$ ，Cubby 中的 `slots` 保存商压缩后的 `payload`，MiniArray 中的 `meta` 保存恢复键所需的变长元数据。查询时，系统不会只依赖 Router 的结果，而会继续用 `payload` 与 `meta` 重建  $\pi(k)$ ，并与当前查询键的置换值进行比较。

---

#### 算法 1 查询过程

---

- 1: 给定查询键  $k$
  - 2: 计算  $g_x \leftarrow \pi(k)$ ，并由  $g_x$  拆分出 Facility 编号  $r$  与 Router 桶号  $b$
  - 3: 在第  $r$  个 Facility 的 `D[b]` 中调用 `LOCATE(k)`
  - 4: 若 Router 返回候选位置  $(C, i)$ ，读取 `C.slots[i]` 与 `C.meta[i]`
  - 5: 根据 `payload` 与 `meta` 重建  $g_x$ ，并与查询键的  $\pi(k)$  比较
- 

本文实现并非对论文中全部微观结构的严格复刻。Router 与 MiniArray 保留紧凑编码和局部管理的接口语义，但更新策略采用整体重建或重打包；k-kick 树

实现为 tail Cubby 内的有界探针与踢出链。因此，本章涉及复杂度和空间指标时均以代码行为为准，第四章实验也只用于说明该原型系统的运行特征。

## 3.2 参数派生与地址拆分

参数模块包含外部配置和内部派生两部分。外部参数位于 TableParams，包括规模提示  $n$ 、k-kick 参数  $k$ 、负载因子 `load_factor` 以及 Feistel 置换种子等。内部参数包括表规模  $N$ 、局部桶数  $K$ 、Facility 数量、Cubby 容量以及最大 tier 数。代码中  $N$  由 `next_pow2` 规整为 2 的幂， $K$  由 `choose_K` 根据  $N$  推导，并限制在可控范围内。

地址拆分由三轮 Feistel 可逆置换开始。对任意键  $k$ ，系统先计算  $g_x = \pi(k)$ ，再取其低  $\log N$  位：

$$g = g_x \wedge (N - 1).$$

由于  $N$  是 2 的幂，上式等价于  $g_x \bmod N$ 。随后按照

$$r = \lfloor g/K \rfloor, \quad b = g \bmod K$$

得到 Facility 编号  $r$  和 Router 桶号  $b$ 。该拆分规则用于插入、查询、删除和迁移后的重新放置，保证所有操作使用一致的路由口径。

---

### 算法 2 键到 Facility 与 Router 桶的拆分

---

- 1: 输入键  $k$ ，当前表状态  $T$
  - 2:  $g_x \leftarrow \pi(k)$
  - 3:  $g \leftarrow g_x \text{ and } (T.N - 1)$
  - 4:  $r \leftarrow \lfloor g/T.K \rfloor$
  - 5:  $b \leftarrow g \bmod T.K$
  - 6: 得到第  $r$  个 Facility 以及其中的  $D[b]$
- 

该拆分依赖两个实现条件。第一，代码中的多处取模操作默认  $N$  与  $K$  为 2 的幂，初始化和扩缩容时必须保持这一不变量。第二，可逆置换不仅用于打散键分布，还参与商压缩场景下的键恢复；若置换种子变化，既有槽位中的 payload 与 meta 将无法按照原上下文恢复出正确键。

### 3.3 Facility 组件

Facility 是系统的局部管理单元，对应代码中的 Facility 结构。其内部主要包含三类状态：按 tier 分组的 Cubby 集合 tiers、唯一尾部 Cubby 相关的 tail\_tier、tail 与 tail\_owned，以及长度为  $K$  的 Router 数组  $D$ 。其中 tiers 使用 `vector<vector<unique_ptr<Cubby>>>` 表示，不同 tier 中 Cubby 的容量不同；tail 是裸指针，指向由 tail\_owned 持有的尾部 Cubby； $D[b]$  负责 Facility 内第  $b$  个局部桶的键到槽位映射。

Facility 的实现重点在于维持局部结构不变量。插入时，系统总是先通过地址拆分定位目标 Facility，再调用 `ensure_tail` 确认该 Facility 存在未满 tail Cubby。如果当前 tail 已满，代码会将旧 tail 挂回对应 tier，并新建一个 tier 0 的 tail Cubby。删除时，如果被删除元素不在 tail 中，系统会从 tail 中搬移一个元素填补空洞，从而尽量保持非 tail Cubby 接近满载。

设计文档中的 Facility 还承担分布不变量和后台重建职责：当某一 tier 的 Cubby 数量显著偏离目标值时，通过拆分或合并 Cubby 恢复比例。当前代码保留 tier、tail 和调度指标等接口，但完整多层重建策略较理论模型有所简化。因此，本文实现中的 Facility 主要承担局部路由聚合、tail 管理和渐进迁移单位三项职责。

### 3.4 Cubby、Slot 与 FreeSlotTree

Cubby 是 Facility 内的基本存储容器，对应代码中的 Cubby 结构。其组成包括 tier、capacity、size、slots、occupied、free\_slots 和 meta。其中 slots 是长度等于容量的 `optional<uint64_t>` 数组，保存商压缩后的 quotient payload；occupied 记录当前被占用的槽位下标，用于扫描、迁移和删除时的尾部回填；meta 是 MiniArray，用于保存每个槽对应的变长 MetaEntry 编码。

Slot 是 Cubby 内的单个整数下标  $i$ 。在逻辑上，槽位内容由 `slots[i]` 与 `meta[i]` 共同决定：前者保存  $\pi(k)$  的高位商，后者保存恢复低位和插入上下文所需的元数据。写入槽位时，系统必须同时更新 payload、MiniArray、occupied、free\_slots 和 Router；删除槽位时，也必须同步清空这些结构。若只更新其

中一部分，后续查询可能出现 Router 指向旧位置、meta 无法解码或键恢复失败等问题。

FreeSlotTree 用于快速定位 Cubby 内空闲槽。它以线性化的树形数组 bitmaps 记录区间空闲数量，叶层宽度  $n$  取不小于容量的最小 2 的幂。构建时，所有合法槽位被视为空闲；调用 mark\_used 或 mark\_free 后，沿父节点向上更新区间计数；调用 find\_free 时，从根节点向下选择仍有空闲计数的子区间，最终得到一个可用槽位。该实现没有采用复杂位运算压缩，而是用较直接的线段树形式换取清晰可靠的维护逻辑。

### 3.5 MiniArray 组件

MiniArray 是 Cubby 内的变长元数据数组，外部接口包括 reset、update、access 和 bits\_total。其内部包含四份状态：bitlens\_ 记录每个条目的位长，offsets\_ 记录每个条目在连续缓冲区中的起始偏移，entries\_ 保存未压缩的条目副本，buf\_ 保存打包后的连续位缓冲。一个 Cubby 初始化时，会根据槽位容量调用 reset，使 MiniArray 条目数与 Slot 数量保持一致。

MiniArray 的更新流程较为直接。调用 update( $i, bits, bitlen$ ) 后，代码先检查下标是否越界，再改写第  $i$  个条目的位长和内容，随后调用 repack()。重打包过程先根据所有 bitlens\_ 重新计算前缀偏移，再分配足够长度的 buf\_，最后逐位把 entries\_ 中的内容写入连续缓冲区。访问某一条目时，系统根据 offsets\_ 和 bitlens\_ 从缓冲区中取回对应比特。

---

#### 算法 3 MiniArray 单次更新流程

---

- 1: 输入槽号  $i$ 、编码比特序列  $bits$ 、位长  $bitlen$
  - 2: 更新 bitlens\_ $[i]$  与 entries\_ $[i]$
  - 3: 根据所有条目位长重新计算 offsets\_
  - 4: 清空并重新分配 buf\_
  - 5: 按新偏移将 entries\_ 中的所有条目逐位写入 buf\_
- 

该组件的指标口径需要单独说明。虽然 buf\_ 是紧凑打包后的位缓冲，但 entries\_ 中仍保留未压缩副本，因此 bits\_total 只能反映逻辑编码位数，不能等同于进程真实内存占用。此外，当前更新方式每次都会重打包整个 Cubby 的元数据，时间代价与槽位数量同阶；这与论文中常数时间动态 MiniArray 的要

求不同，但便于检查编码结果并统计逻辑位数。

### 3.6 Router 组件

Router 是 Facility 内的局部查询路由器，负责维护从完整键到 (Cubby\*, slot) 的映射。其公开接口包括 locate、contains、insert、erase、clear 和 bits\_total。内部状态包括 entries\_、nodes\_、buckets\_、root\_ 和 encoded\_bits\_。其中 Entry 保存原始键、目标位置以及由键生成的 64 位随机比特串；Node 保存左右孩子、叶子 entry 下标、碰撞桶下标和分裂位 split\_bit。

Router 不直接按键值大小建树，而是通过 splitmix64(key  $\oplus$  常量) 得到稳定的 64 位伪随机比特串。插入新键时，若 entries\_ 中已存在相同键，则覆盖其目标位置；否则追加新 entry。删除时，代码从 entries\_ 中移除对应键。无论插入还是删除，Router 随后都会调用 rebuild()，重新计算所有 entry 的比特串，构造 Patricia 风格二叉 trie，并重新统计编码位数。

查询时，从根节点出发，依据查询键的随机比特串和结点的 split\_bit 选择左右子树。若到达普通叶结点，则比较原始键是否相同；若到达碰撞桶，则在桶内线性扫描。函数同时返回访问步数，供指标系统累计为 router\_probe\_steps。

---

#### 算法 4 Router 查询流程

---

```
1: 输入键  $k$ 
2:  $bits \leftarrow \text{KEY\_BITS}(k)$ ,  $steps \leftarrow 0$ ,  $u \leftarrow \text{root}$ 
3: while  $u$  为合法结点 do
4:    $steps \leftarrow steps + 1$ 
5:   if  $u$  为碰撞桶结点 then
6:     在桶内逐项比较原始键，若相等则返回对应位置与  $steps$ 
7:     若桶内不存在该键，则返回未命中与  $steps$ 
8:   else
9:     根据  $bits$  在 split_bit 处的取值进入左子树或右子树
10:  end if
11: end while
12: 返回未命中与  $steps$ 
```

---

Router 的更新成本和空间统计也需要按实现口径解释。当前实现没有保存真实序列化 bitstream，而是用 BitWriter 遍历树结构估算编码位数；同时，插入和删除后会整体重建 trie。因此，bits\_total 可用于观察路由结构的逻辑

规模，但不能代表真实内存；Router 查询路径通常较短，更新路径则不具备论文级 Local Query Router 的常数时间动态更新保证。

### 3.7 MetaEntry 与商压缩

商压缩部分由 MetaEntry、编码布局 and 槽内 payload 共同完成。槽内 `slots[i]` 保存  $\pi(k)$  去掉低  $\log N$  位后的高位，即 `quotient_payload`。与低位恢复有关的信息保存在 MetaEntry 中，主要字段包括 `delta`、`mid_bits`、`router_bits` 和 `insert_bits`。`delta` 表示 Cubby 内偏好位置与实际槽位之间的差值，`mid_bits` 保存从局部槽位恢复到 Router 桶范围所需的中间位，`router_bits` 是路由相关指纹字段，`insert_bits` 记录插入过程中的步数标签。

设待处理键为  $X$ ，经过双射哈希后得到  $g_x = \pi(X)$ 。代码首先将  $g_x$  拆成两部分：高于  $\log N$  的剩余位和低  $\log N$  位。前者在实现中称为 `payload`，也可理解为商压缩中的 `rest` 部分，直接写入 `slots[i]`；后者记为  $g^*$ ，用于确定元素在表中的路由位置。随后  $g^*$  继续被拆分为 Facility 编号  $r$ 、局部桶内编号  $g_k$ ，而  $g_k$  又被拆分为中位 `mid_bits` 与 Cubby 内偏好槽位  $g_i$ 。图 3-1 给出了这一位段关系。

| rest / payload           | Facility 高位 $r$         | 中位 mid_bits             | 低位 $g_i$                  |
|--------------------------|-------------------------|-------------------------|---------------------------|
| $g_x \gg \log N$         | $\lfloor g^*/K \rfloor$ | $\lfloor g_k/I \rfloor$ | $g_k \bmod I$             |
| 存入 <code>slots[i]</code> | 选择 Facility             | 恢复局部桶位置                 | 结合槽位计算 <code>delta</code> |

图 3-1  $\pi(X)$  在当前实现中的位段用途

上述符号与代码实现的对应关系如下。令  $g^* = g_x \text{ and } (N-1)$ ，则  $r = \lfloor g^*/K \rfloor$ ， $g_k = g^* \text{ and } (K-1)$ 。在给定 Cubby 容量  $|I|$  后，代码计算  $g_i = g_k \bmod |I|$ ，并将实际槽位  $i$  与偏好槽位  $g_i$  的差值编码为  $\text{delta} = g_i - i$ 。因此，低位  $g_i$  并不直接保存，而是通过 `delta` 与槽下标  $i$  间接恢复；中位 `mid_bits` 被保存为  $g_k/|I|$ ；`rest` 部分以 `payload` 形式保存在槽数组中。恢复时，系统按  $g_i = i + \text{delta}$ 、 $g_k = \text{mid\_bits} \times |I| + g_i$ 、 $g^* = r \times K + g_k$ 、 $g_x = (\text{payload} \ll \log N) | g^*$  的顺序重建完整置换值。

写入槽位时，系统调用 `make_meta_entry` 根据键、 $\pi(k)$ 、槽位下标、Facility 编号、 $N$ 、 $K$ 、Cubby 容量和插入步数构造元数据，再调用 `encode_meta_entry` 按 `MetaCodecLayout` 编码为比特序列，最后通过 `MiniArray` 写入对应槽位。

恢复时，系统先从 MiniArray 解码得到 MetaEntry，再结合 Facility 编号、槽位下标、Cubby 容量、payload 和  $\log N$  调用 `reconstruct_gx_pi`，重建完整的  $\pi(k)$ 。若需要得到原始键，则继续调用 Feistel 逆置换。

---

**算法 5** 从槽位恢复原始键

---

```

1: 输入 Facility 编号  $r$ 、Cubby  $C$ 、槽下标  $i$ 、表状态  $T$ 
2: if  $i$  越界或  $C.slots[i]$  为空 then
3:   返回空结果
4: end if
5: 从  $C.meta[i]$  解码得到 MetaEntry
6: if 元数据解码失败 then
7:   返回空结果
8: end if
9:  $g_x \leftarrow \text{RECONSTRUCT\_GX\_PI}(r, T.N, T.K, C.capacity, i, payload, MetaEntry)$ 
10: if  $g_x$  无法恢复 then
11:   返回空结果
12: end if
13: 返回  $\pi^{-1}(g_x)$ 

```

---

商压缩恢复依赖完整上下文，包括 Facility 编号、表规模  $N$ 、局部桶数  $K$ 、Cubby 容量和槽位下标。扩缩容迁移后，元素必须按新表状态重新写入 payload 与 meta，不能直接复制旧槽位内容。查询阶段还必须用  $\pi(k)$  校验恢复结果，以避免 Router 指针错误或元数据损坏导致误判命中。

### 3.8 插入路径与简化 k-kick

插入入口为 `insert`。该函数在互斥锁保护下调用 `insert_no_lock`，然后根据结果触发扩缩容检查、后台重建预算和迁移预算。主流程首先检查表是否初始化，再计算  $g_x = \pi(k)$ ，通过算法 2 定位 active 表中的 Facility 和 Router 桶。若 active 表已存在该键，则返回幂等的未插入结果；若处于迁移状态，还需要在 old 表中做同样检查。确认键不存在后，系统调用 `ensure_tail` 准备 tail Cubby，再调用 `kkick_insert` 写入。

当前工程版 k-kick 并没有实现论文中跨多层 Cubby 的完整驱逐树，而是在单个 tail Cubby 内执行有界探针与踢出链。探针位置由  $\pi(cur)$ 、步数和 `split-mix64` 混合后对 Cubby 容量取模得到。若探针位置为空，系统直接写入 payload 和 meta，更新 `occupied`、`free_slots` 与 Router；若探针位置已被占用，系统

先从该槽恢复旧键,删除旧键在 Router 中的记录,再用当前键覆盖该槽,并把被踢出的旧键作为下一轮待插入键。若经过  $k$  轮仍未找到空位,则通过 FreeSlot-Tree 选择任意空槽作为兜底位置。

---

**算法 6** tail Cubby 内的有界踢出插入

---

```

1: 输入待插入键  $key$ 、tail Cubby  $C$ 、Facility 编号  $r$ 
2:  $cur \leftarrow key$ ,  $moved \leftarrow 0$ 
3: for  $step = 0, 1, \dots, k$  do
4:    $pos \leftarrow \text{PROBE\_SLOT}(cur, C.capacity, step)$ 
5:   if  $C.slots[pos]$  为空 then
6:     写入  $cur$  的 payload 与 meta, 并维护空闲槽和 Router
7:     返回成功及移动次数  $moved$ 
8:   else
9:     从  $pos$  恢复被踢出的键  $kicked$ 
10:    if 恢复失败 then
11:      返回失败
12:    end if
13:    删除  $kicked$  的旧路由, 用  $cur$  覆盖该槽并登记新路由
14:     $cur \leftarrow kicked$ ,  $moved \leftarrow moved + 1$ 
15:  end if
16: end for
17: 通过 FreeSlotTree 选择空槽写入  $cur$ , 并返回成功

```

---

该路径对维护顺序有明确要求。发生踢出时,必须先恢复旧键并删除旧路由,再覆盖槽位和登记新路由,否则 Router 可能短暂指向错误键。由于当前实现只在 tail Cubby 内踢出,实验中的 `kick_count` 反映的是工程版移动次数,而不是理论  $k$ -kick 树的跨层交换次数。

### 3.9 查询与删除路径

查询路径是只读路径。系统计算  $\pi(k)$  后定位 active 表中的 Facility 和 Router 桶,调用 `Router::locate` 获取候选位置,再调用 `slot_pi_matches` 解码 meta、重建  $\pi(k)$  并比较。若 active 表未命中且当前存在 old 表,则对 old 表重复上述步骤。查询不会修改结构,也不会推进迁移预算。

删除路径首先执行与查询类似的定位过程。若命中,系统清空 `slots[slot]`,将对应 meta 更新为空条目,从 occupied 删除槽位,调用 `mark_free` 标记空闲,并从 Router 中删除该键,同时递减元素计数。如果删除位置不属于 tail Cubby,系统会从 tail 中取出一个元素,恢复其原始键后搬移到刚刚空出的槽位,

并更新 payload、meta 和 Router。该策略用于维持非 tail Cubby 尽量满载的结构习惯。

---

**算法 7 查询命中判定**

---

```
1: 输入查询键  $k$ 
2: 在 active 表中定位  $(r, b)$ , 调用  $D[b].locate(k)$ 
3: if Router 返回候选槽位 then
4:   调用 slot_pi_matches 校验候选槽位
5:   根据校验结果返回命中或未命中
6: end if
7: if active 表未命中且 old 表存在 then
8:   在 old 表中重复定位与槽位校验
9: end if
10: 返回最终查询结果
```

---

查询和删除需要注意迁移窗口内的双表语义。插入只向 active 表写入，但查询和删除必须同时考虑 active 表和 old 表。删除时如果同一键位于 old 表中，也需要在 old 表执行删除，以避免迁移后旧键重新出现。

### 3.10 扩缩容与双表渐进迁移

系统通过 maybe\_resize\_locked 判断是否需要扩缩容。当元素数超过  $load\_factor \times N$  时，目标容量设为  $2N$ ；当元素数非零且低于  $N/4$  时，目标容量设为  $\max\{2, N/2\}$ 。触发扩缩容后，start\_migration 将当前 active 表移动到 old\_，再根据新的  $N$  和  $K$  构造空 active 表。此时元素总数  $n_$  不变，后续新写入只进入 active 表。

迁移由 run\_migrate\_budget 渐进推进。每次插入或删除结束后，系统至多处理一个旧 Facility。处理时不依赖旧 Router，而是扫描旧 Facility 的 tail 和各 tier 中所有占用槽位，调用算法 5 恢复原始键，再按新表参数重新定位并插入 active 表。若某个键已经能在 active 表中找到，则说明它已经被新写入或先前迁移处理过，迁移过程会跳过该键。迁移完成一个 Facility 后，旧 Facility 的 Router、tiers 和 tail 会被清空，迁移进度向后推进。

该机制的优点是避免扩缩容时一次性重哈希全部元素，从而降低单次操作的阻塞时间。其限制也较明确：查询操作不会消耗迁移预算，如果扩缩容后长期没有插入或删除，old 表会继续保留。此外，迁移必须通过键恢复和重新插入完

---

**算法 8** 单次迁移预算

---

```
1: 若 old_ 不存在, 则结束本次迁移
2: 取迁移进度指向的旧 Facility
3: 扫描该 Facility 的 tail 与各 tier, 恢复其中所有占用键
4: for 每个恢复出的键  $x$  do
5:   if  $x$  已存在于 active 表 then
6:     跳过该键
7:   else
8:     按新表参数定位 Facility, 调用 ensure_tail 与 kkick_insert
9:   end if
10: end for
11: 清空该旧 Facility, 并推进迁移进度
12: 若所有旧 Facility 均已处理, 则释放 old_
```

---

成, 不能直接复制旧 payload 与 meta, 因为新表的  $N$ 、 $K$ 、Facility 编号和 Cubby 容量都可能已经改变。

## 第四章 系统验证与实验设计

### 4.1 实验目标与评价口径

本章验证第三章实现的核心路径，包括功能正确性、操作延迟、累计指标和结构快照。由于当前系统是理论结构的工程原型，实验不用于替代第二章中的概率分析，也不用于证明空间最优性；实验目标限定为三个问题：插入、查询、删除和重复插入是否满足预期语义；Router、MiniArray、商压缩和工程版 k-kick 对延迟表现有何影响；运行过程中记录的移动次数、路由步数和元数据位数能否反映实现中的主要开销。

### 4.2 功能正确性测试

otsh\_tests 是系统的自动化回归入口。该程序直接调用 HashTable，不经过 HTTP 层和 JSON 编解码流程，测试对象集中在哈希表核心结构。测试流程包括：

- 初始化哈希表，并检查返回状态与表状态中的 *N*、Facility 数量是否有效；
- 使用固定随机种子生成唯一键集合，执行批量插入，要求每个新键返回 `inserted=true`；
- 对全部已插入键执行查询，要求返回 `found=true`；
- 随机删除一半键，要求删除操作成功；
- 再次查询已删除键，要求返回未命中；
- 对剩余键重复插入，要求返回 `inserted=false`，验证幂等语义。

测试结束时输出单行 JSON，字段包括 `all_passed`、通过和失败数量、各测试用例名称与结果，以及 `ops_insert`、`ops_query`、`ops_delete`、`insert_moved_total`、`delete_moved_total` 等摘要指标。结果写入本地日志。

### 4.3 性能基准

otsh\_perf 用于测量内存态哈希表路径。该程序不链接 MySQL，也不经过 HTTP JSON 编解码，避免存储层和接口层影响插入、查询、删除本身的耗时。程序支持命令行参数 `n_insert`、`n_rand_queries`、`table_n_hint`，并通过环境变量 `OTSH_SEED` 固定随机种子。

每个阶段记录操作数、吞吐、平均延迟与 p50 分位数。结果输出到 `stderr`，单行 JSON 输出到 `stdout`，便于脚本收集到 `perf/results/`。本文实验参数为 `n_insert = 5 × 104`、`n_rand_queries = 2 × 105`、`table_n_hint = 2 × 105`、`OTSH_SEED = 1`。

表 4-1 otsh\_perf 微基准真实运行结果

| 阶段             | 操作数    | 总耗时/ms  | 吞吐/s <sup>-1</sup> | 平均/ns | p50/ns |
|----------------|--------|---------|--------------------|-------|--------|
| 初始化            | —      | 35.437  | —                  | —     | —      |
| 插入             | 50000  | 1779.45 | 28099              | 35455 | 10700  |
| 顺序查询（全命中）      | 50000  | 146.23  | 341916             | 2802  | 2700   |
| 随机查询（命中）       | 200000 | 637.76  | 313600             | 2981  | 2900   |
| 删除一半键          | 25000  | 162.40  | 153939             | 6372  | 5800   |
| 混合查询（命中 + 未命中） | 100000 | 258.93  | 386199             | 2242  | 2600   |

表中数据与第三章的路径分析一致。插入阶段平均延迟为 35455 ns，p50 为 10700 ns，二者差距较大，说明延迟分布右偏，存在少数较慢路径；整体仍显著高于查询路径。原因在于插入除置换、Router 定位和槽位写入外，还会维护 MiniArray、Router、FreeSlotTree，并可能发生有限踢出或扩缩容迁移。顺序命中查询与随机命中查询的平均延迟分别为 2802 ns 和 2981 ns，p50 与平均接近，说明查询主要受局部 Router 查找和槽位校验影响。删除阶段平均延迟为 6372 ns，高于查询但低于插入，符合其需要清理槽位、维护 Router 并可能执行 tail 回填的实现路径。混合查询平均延迟为 2242 ns，未出现全表扫描带来的明显抬升。

同一次运行结束后，程序输出哈希表状态和累计指标，结果见表 4-2。此时系统保留 25000 个元素，active 表规模为 65536， $K = 256$ ，Facility 数量为 256。插入移动次数累计为 46941，最大单次移动次数为 5；删除移动次数累计为 376，最大单次移动次数为 1。该结果表明，当前工程版 k-kick 插入确实发生了踢出，但单次移动次数在本次运行中保持较小。Router 访问步数最大值为 5，说明在该参数和随机种子下，局部 Patricia 路由未形成较深路径。

表 4-2 otsh\_perf 运行结束后的状态与累计指标

| 指标                         | 数值        |
|----------------------------|-----------|
| 当前元素数 $n$                  | 25000     |
| 表规模 $N$                    | 65536     |
| Router 桶数 $K$              | 256       |
| Facility 数量                | 256       |
| insert_moved_total         | 46941     |
| insert_moved_max           | 5         |
| delete_moved_total         | 376       |
| delete_moved_max           | 1         |
| router_steps_total         | 642741    |
| router_steps_max           | 5         |
| meta_bits_total            | 633735932 |
| meta_bits_max              | 16365     |
| resize_start/resize_finish | 8/8       |
| 平均上报元数据位数/插入               | 12674.72  |

## 4.4 指标与数据库观测

操作级指标以 SqlMetricRow 形式产生,核心字段包括 operation\_type、probe\_count、kick\_count、latency\_ns、cubby\_tier 和 snapshot\_id。存储层先把指标放入内存缓冲,再批量写入 otsh\_op\_metrics,避免高频操作逐条落库造成过大扰动。

这些字段与第二章中的探针复杂度和交换成本相对应。probe\_count 记录局部 Router 的访问步数, kick\_count 记录更新过程中的移动次数, latency\_ns 记录工程延迟, cubby\_tier 用于区分不同层级 Cubby 上的操作行为。后续分析可以按操作类型统计延迟分位数,也可以按 tier 或快照比较路由步数和移动次数。

结构快照由 HashTable::visit\_structure 驱动。系统将 Facility、Cubby、Slot 和 tier 聚合信息写入相应数据表: facility 记录 tail 所在 Cubby, cubby 记录 tier、capacity、size 和 is\_tail, slot\_snapshot 记录槽位是否占用及  $\pi(x)$  形式的 key hash, tier\_stat 记录每个 Facility 在各 tier 上的 Cubby 数量。这些表可用于检查 tail 是否唯一、非 tail Cubby 是否接近满载、tier 数量是否偏离目标分布。

指标写入属于观测路径,不属于哈希表核心算法。性能基准程序绕开 MySQL 和 HTTP 层,是为了减少观测系统对热路径延迟的干扰;数据库指标更适合用于实验后分析、结构审计和可视化展示。

## 第五章 总结与展望

### 5.1 工作总结

本文围绕 *On the Optimal Time/Space Tradeoff for Hash Tables* 中的主要结构, 完成了一个 C++ 开放寻址哈希表原型。系统对外以 `HashTable` 为入口, 内部按 `Facility`、`Cubby`、`Slot` 三层组织元素; 通过三轮 Feistel 可逆置换生成键的随机像; 使用 `Router` 维护局部键到槽位的映射; 在槽位中保存商压缩后的 `payload`; 通过 `MiniArray` 保存恢复  $\pi(k)$  所需的 `MetaEntry`; 扩缩容时采用 `active/old` 双表渐进迁移。

功能测试覆盖了初始化、插入、查询、删除、删除后再次查询以及重复插入等路径。性能测试: 在 `n_insert=50000`、`n_rand_queries=200000`、`table_n_hint=200000`、`OTSH_SEED=1` 的条件下, 插入平均延迟为 35455 ns, 顺序命中查询和随机命中查询平均延迟分别为 2802 ns 和 2981 ns, 删除一半键的平均延迟为 6372 ns。运行结束时, 表中保留 25000 个元素,  $N = 65536$ 、 $K = 256$ , 插入移动次数累计为 46941, `Router` 最大访问步数为 5。上述结果与第三章的实现路径相符: 查询主要执行局部路由与槽位校验, 插入还需要维护 `MiniArray`、`Router`、空闲槽结构, 并可能发生有限踢出。

### 5.2 不足与局限

本文实现与论文中的完整理论结构仍有差距。第一, `k-kick` 机制没有实现为跨多层 `Cubby` 的完整分层驱逐结构, 而是在 `tail Cubby` 内执行有界探针和踢出链; 因此, 实验中的 `kick_count` 只能反映当前工程路径中的移动次数, 不能等同于理论 `k-kick` 树的交换成本。第二, `MiniArray` 和 `Router` 保留了变长编码、局部路由和位数统计的接口语义, 但更新时分别采用整体重打包和整体重建, 尚未达到论文中常数时间局部更新的要求。第三, `meta_bits_total`、`Router` 编码位数等指标是逻辑编码规模或按操作累计值, 未计入 `std::vector`、指

针、对象头等实际运行时开销，因此不能作为严格空间上界证明。

实验范围也有限。本文只采用固定随机种子下的单机微基准，并使用保存的一组大规模运行日志作为主要数据来源；尚未系统改变  $n$ 、 $k$ 、负载因子和随机种子，也没有进行多轮取中位数或跨机器复现实验。本文也没有与标准库哈希表或其他开源哈希表实现并列比较。因此，实验结论仅能说明该原型在当前参数与实现策略下的行为，不能推广为通用哈希表性能结论。

### 5.3 可改进方向

后续工作应围绕上述差距展开。首先，可将当前 `tail Cubby` 内的有界踢出逐步替换为更接近论文描述的分层 `k-kick` 维护，使 `kick_count` 与理论交换成本具备更明确的对应关系。其次，可改造 `MiniArray` 和 `Router` 的更新方式，减少整段重打包和整棵 `trie` 重建带来的更新开销，并重新区分逻辑编码位数与真实内存占用。最后，可将性能实验整理为固定参数、固定机器说明、多轮运行和结果汇总的脚本，在此基础上再比较不同  $k$ 、负载因子和表规模下的变化。

## 参考文献

- [1] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. On the Optimal Time/Space Tradeoff for Hash Tables[EB/OL]. (2021-11-04) [2026-05-02]. <https://arxiv.org/abs/2111.00602>. arXiv: 2111.00602.
- [2] CORMEN T H, LEISERSON C E, RIVEST R L, et al. Introduction to Algorithms [M]. 3rd ed. Cambridge, MA: MIT Press, 2009.
- [3] PAGH R, RODLER F F. Cuckoo Hashing[J/OL]. Journal of Algorithms, 2004, 51(2): 122-144. DOI: 10.1016/j.jalgor.2003.12.002.
- [4] FREDMAN M L, KOMLÓS J, SZEMERÉDI E. Storing a Sparse Table with  $O(1)$  Worst Case Access Time[J/OL]. Journal of the ACM, 1984, 31(3): 538-544. DOI: 10.1145/828.1884.

## 致 谢

行文至此，落笔为终。感谢遇到的导师、感谢身边的每一个同学、感谢遇到的每一个人。凡是过往，皆为序章，很喜欢看到的一句话，人生如棋，落子无悔。愿我们跃入人海，各自灿烂。